

4×4 Systolic Array Accelerator

Group 2

Anthony Kodjo
akodjo@umich.edu

Abrar Khan
abrark@umich.edu

Abraham Kallivayalil
abkalliv@umich.edu

Viraj Shah
virajsh@umich.edu

Rishabh Bhattarai
brishabh@umich.edu

Lourd Louis
lmaem@umich.edu

Abstract—This project presents the design and implementation of a 4×4 systolic array accelerator integrated into a custom 16-bit RISC processor datapath to efficiently perform matrix multiplication-style computations. The system extends a baseline processor by introducing a specialized hardware unit composed of processing elements (PEs) that execute multiply-accumulate (MAC) operations in a spatially parallel and pipelined manner. Data is fed into the array using a controller-driven scheduler that ensures correct temporal alignment of operands, enabling accurate computation within the time-skewed nature of systolic architectures. To support data movement and storage, a 16×16 SRAM module is incorporated as an intermediate buffer between the systolic array and the register file, forming a circular datapath that allows computed results to be reused for subsequent operations. Additional custom instructions, including `SYSOP`, `DRAIN`, and `SYSLOAD`, are implemented to control multi-cycle systolic execution, data movements, and memory interaction. The design also includes output buffering and time-multiplexing techniques to efficiently handle limited I/O bandwidth when extracting results from the array. The entire system is implemented using a full digital design flow RTL-to-GDSII and post-APR (automatic place-and-route) verification. This enables evaluation of physical design constraints such as delay, routing complexity, and signal synchronization. Compared to sequential execution on a standard ALU, a systolic architecture demonstrates the advantages of spatial parallelism, including significantly improved throughput, efficient data reuse, and scalable module design.

I. INTRODUCTION

Matrix multiplication is a fundamental operation in many modern computing workloads, including machine learning, signal processing, and scientific computing. These applications require a large number of repeated arithmetic operations, which can become a performance bottleneck when executed on conventional sequential processors. In a baseline 16-bit RISC processor, operations such as matrix multiplication must be performed over multiple cycles using limited arithmetic resources, resulting in low throughput and inefficient utilization of hardware.

To address this limitation, specialized hardware accelerators are commonly used to exploit parallelism at the architectural level. One widely used approach is the systolic array [3], a spatial computing architecture composed of a grid of simple processing elements that communicate locally with their neighbors. Each PE performs a small portion of the overall computation, allowing data to flow through the array in a pipelined manner. This structure enables high throughput, efficient data reuse, and reduced reliance on long-distance data movement, making systolic arrays well-suited for matrix-based computations [1].

In this project, a 4×4 systolic array accelerator is integrated into a custom 16-bit RISC processor datapath to enable efficient matrix multiplication-style operations. The design extends the baseline processor by introducing a dedicated compute unit along with additional control logic and instruction set extensions to support multi-cycle execution and data movement. A key component of the system is the inclusion of an SRAM module, which serves as an intermediate buffer between the systolic array and the register file, enabling structured data storage and facilitating a circular datapath in which computed results can be reused for subsequent operations.

A major challenge in systolic array design is ensuring correct temporal alignment of data as it propagates through the array. Since computations are time-skewed, input operands must be carefully staged so that the appropriate values arrive at each processing element simultaneously. This project addresses this challenge through a controller-driven data feeding mechanism that utilizes arrays of register buffers and provides control over input timing.

The complete system is implemented using a full digital design flow, including behavioral Verilog simulation and post-layout verification after automatic place-and-route. This allows evaluation of both functional correctness and physical design constraints such as delay, routing complexity, and signal synchronization. Through this work, the project aims to demonstrate how architectural specialization and hardware parallelism can significantly improve performance for computation-intensive tasks compared to purely sequential processor designs.

II. DESIGN AND IMPLEMENTATION

The final design extends the baseline 16-bit RISC processor with a 4×4 systolic array accelerator, a unified controller, and a dual-port SRAM to support matrix multiplication computation. Over the course of the development, the architecture evolved from an initial standalone concept into a fully integrated datapath in which data flows from the register file of the baseline processor to the controller, through the systolic array, into SRAM, and then back into the register file through a `SYSLOAD` operation. This circular datapath was an important part of the final system because it allowed computed results to be stored and reused for later operations rather than treating the array as an isolated block. The project's datapath is visualized in Figure 1.

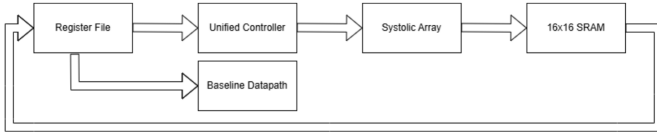


Fig. 1. Abstracted diagram of new datapath.

A. Processing Element and Systolic Array

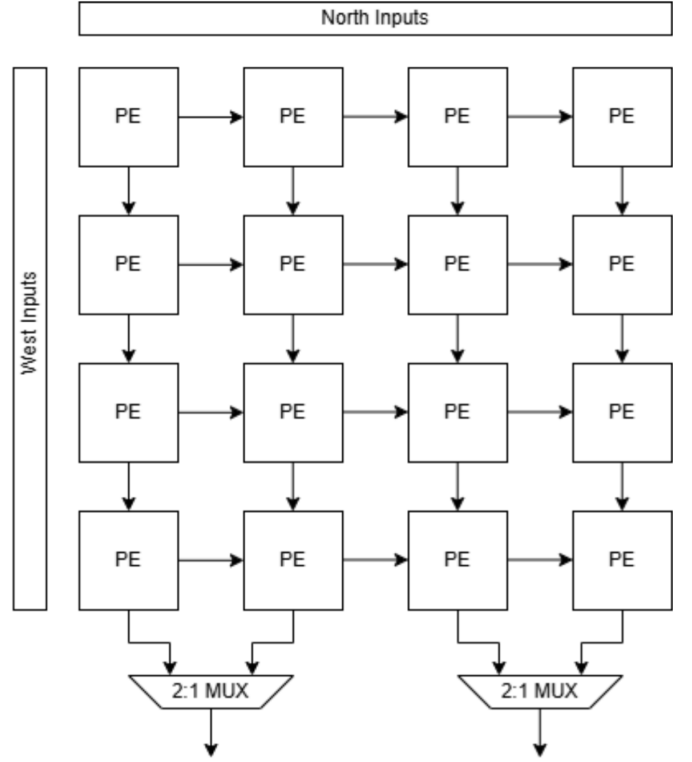
At the core of the accelerator is a 4×4 grid of processing elements (PEs). Each PE accepts two 4-bit signed values from both the north and west directions, multiplies them, and accumulates the product into a signed result register. Each PE forwards its west input to the east and its north input to the south so that operands can propagate through neighboring cells. The regular interconnect structure makes the array modular and suitable for replication. The PE also includes a drain mechanism that enables accumulated results to shift downward along each column during result extraction. This allows values to be collected from the bottom of the array using just two multiplexers, reducing hardware complexity compared to directly accessing each PE output individually.

The systolic array module instantiates sixteen copies of the PE cell and connects them by row and column. West inputs enter the left edge and propagate horizontally, while north inputs enter from the top and propagate downwards through columns. Partial sums move downwards through the array by feeding each row from the results of the row above, and the final outputs are taken from the bottom row. In the implemented design, the bottom-row accumulators produce four final 10-bit signed results at a time, which are then sign-extended to 16 bits before being sent to the SRAM module. A visualization of the array is shown in Figure 2.

B. Input Staging and Data Alignment

A major design challenge was ensuring that the correct operands arrived at each processing element at the correct cycle. Because systolic computation is time-skewed, valid matrix multiplication behavior depends on temporal alignment rather than simultaneous entry of all inputs. Initially, a FIFO approach was used to stage inputs; however, this was then changed to using shift registers to stage the inputs inside the unified controller. This change was made to reduce both logic and hardware complexity as well as overall area, as shift register implementations tend to have a lower overall area than FIFOs.

The controller accepts two 16-bit words from the register file, where each word packs four signed 4-bit values. It then unpacks these values into the four west lanes and four north lanes that feed the array. The first lane (the top-left cell) is injected directly, while the remaining lanes pass through one-, two-, and three-cycle delay registers so the higher-index operands enter the array later. This creates the diagonal staggering required for correct data flow through the grid.



D. Output Multiplexing and Register File Interface

An important implementation issue was the mismatch between the number of array outputs needed and the limited writeback bandwidth available in a given cycle. The systolic computation produces sixteen result values; however, only a subset can be written out at once. The final design addresses this by using a time-multiplexed drain path controlled by the MUXes shown at the bottom of Figure 2. On one drain phase, the system selects two of the bottom-row outputs; on the alternate phase, it selects the other two. Intermediate registers were added to hold delayed versions of selected outputs so that data remained aligned with the writeback control timing.

Each 16-bit register stores four packed 4-bit signed values, corresponding to elements of a matrix row or column. This packing allows efficient utilization of the datapath bandwidth, as a single register read provides multiple operands to the array. During the FEED stage, the controller selects register entries and streams their contents into the array input lanes, reducing the number of required memory accesses and aligning naturally with the four-lane structure of the array inputs.

E. Dual-Port SRAM

A dual-port SRAM is used to support writing multiple results during the DRAIN phase and sequential reading during SYSLOAD. During result extraction, the controller generates the appropriate addresses and write-enable signals to store computed values across multiple cycles. This buffering is necessary because the array produces outputs over time, while the processor expects results in 16-bit register format. During SYSLOAD, the controller increments the SRAM address and writes each stored value back into the register file, enabling the circular datapath.

III. RESULTS AND CONCLUSION

The final system demonstrates a complete hardware implementation of a systolic array accelerator integrated into a 16-bit RISC processor datapath. The design supports multi-cycle execution of matrix-style computations using custom instructions (SYSOP, DRAIN, and SYSLOAD), enabling data to flow through a circular datapath consisting of the register file, controller, systolic array, and SRAM.

A. Performance

Performance was evaluated using a 10 ns clock period. From the start of the systolic operation to the final output, the total latency is approximately 38 clock cycles. The FEED phase requires 4–5 cycles to propagate inputs to the array edges, followed by 10 cycles of COMPUTE time as data propagates through the array. The DRAIN phase requires 4 cycles to shift results downwards through the columns, and the SYSLOAD phase produces all 16 outputs over 15 cycles, achieving a throughput of approximately one result per cycle once the pipeline is active.

To provide a comparison against a baseline implementation, the same 4×4 matrix multiplication was estimated using only the original ISA without the systolic array. Each output

requires four MAC operations, resulting in 64 total operations. With a 2-stage pipeline, this results in an estimated 256 cycles for computation, and including additional overhead for loop control and data movement, the total execution time is estimated to be within 300 to 350 cycles. Compared to the 38-cycle execution of the systolic array, this represents an approximate $8 \times$ speedup, further demonstrating the advantages of spatial parallelism and hardware acceleration.

B. Power

Power data was collected from post-synthesis Design Compiler reports (*.dc.rpt) for each module. Because report_power was not invoked in the Innovus APR flow, these figures are pre-layout estimates. The top-level synthesis treats the systolic array, controller, and SRAM as black-box IP blocks, so its reported 0.681 mW represents only the glue-logic power; sub-module totals must be summed separately to obtain the full system power.

Table I summarizes total power per module at its target clock. Switching and internal power contributions are: PC (0.123/0.341 mW), systolic array (0.294/3.036 mW), controller (0.070/1.667 mW), and top-level glue (0.081/0.599 mW). Leakage is $3.99 \mu\text{W}$ (PC), $34.6 \mu\text{W}$ (systolic array), and $12.9 \mu\text{W}$ (controller).

TABLE I
PER-MODULE TOTAL POWER FROM POST-SYNTHESIS REPORTS.

Module	Clock (MHz)	Total (mW)
Program Counter	1333	0.467
Systolic Array	667	3.365
Controller	1000	1.750
Top-level (glue)	667	0.681

For the SRAM (tsdn651pa16x16m4f), power characteristics were extracted from the liberty file and datasheet at the TT, 1.2 V, 25 °C corner. The cell leakage power is $0.744 \mu\text{W}$, and the dynamic current scales linearly with frequency: total read current = $7.097 \times F \mu\text{A}$ and total write current = $5.791 \times F \mu\text{A}$, where F is the clock frequency in MHz. At the top-level operating frequency of 667 MHz this yields approximately 5.68 mW of dynamic read power and 4.63 mW of dynamic write power per port ($P = V_{DD} \times I$, with $V_{DD} = 1.2 \text{ V}$).

Table II summarizes the estimated total system power, excluding SRAM dynamic power since it depends on the read/write activity ratio at runtime.

C. Area

Area results were obtained from the post-layout designs of each module. The systolic array itself occupies approximately $13,845 \mu\text{m}^2$ ($117.9 \mu\text{m} \times 118.2 \mu\text{m}$), the SRAM occupies $9,852 \mu\text{m}^2$ ($53.33 \mu\text{m} \times 184.75 \mu\text{m}$), and the unified controller occupies $5,476 \mu\text{m}^2$ ($74 \mu\text{m} \times 74 \mu\text{m}$). The complete post-layout integration, including the systolic array accelerator and the 16-bit baseline processor, occupies a total area of approximately $132,219 \mu\text{m}^2$ ($482.2 \mu\text{m} \times 274.2 \mu\text{m}$). From

TABLE II
ESTIMATED TOTAL SYSTEM POWER (STATIC + SWITCHING; SRAM
DYNAMIC EXCLUDED).

Module	Total Power (mW)
Program Counter	0.467
Controller	1.750
Systolic Array	3.365
SRAM (leakage only)	0.001
Top-level glue logic	0.681
Grand Total	≈ 6.26

these results, the system demonstrates a clear tradeoff between area and performance. While the addition of the accelerator adds a meaningful amount of area, it provides a significant improvement in computational throughput compared to sequential implementations of similar operations.

D. Lessons Learned

Several key lessons were learned throughout the project. First, design decisions can evolve significantly throughout the development process. For example, the input staging mechanism initially began as a FIFO-based design, but after further discussion and evaluation of timing and control complexity, it was replaced with a shift-register-based approach that provided more precise control and reduced overhead. This highlights the importance of flexibility and iterative design in hardware development.

Additionally, the most challenging aspect of the project was not the systolic array itself, but the design of the unified controller. While individual modules such as the processing elements can be designed and verified independently, integrating them into a complete system requires careful coordination of control logic, timing, and dataflow. The controller design involved implementing a finite state machine and managing interactions between the register file, systolic array, and SRAM, which introduced significantly more complexity. This emphasizes that system-level design and computer architecture considerations are often more difficult than implementing isolated functional blocks.

REFERENCES

- [1] M. Bastiaan, "Building Systolic Arrays with Clash," *clash-lang*, Jul. 25, 2018. <https://clash-lang.org/blog/0002-systolic-arrays/> (accessed Apr. 25, 2026).
- [2] Onur Mutlu Lectures, "Computer Architecture – Lecture 28: Systolic Array Architectures (Fall 2023)," *YouTube*, Feb. 01, 2024. <https://www.youtube.com/watch?v=XkgtANeDrm8> (accessed Apr. 25, 2026).
- [3] K. Berger, A. Damle, and J. Retcher, "Construction of a 4x4 Systolic Array," 2019, Accessed: Apr. 25, 2026. [Online]. Available: https://www.a-damle.com/project_files/systolic_array/systolic_array.pdf
- [4] J. Miller, "Create a Finite State Machine." <https://jimmyhmler.com/fsm-maker/> (accessed Apr. 25, 2026).
- [5] T. Raja, "Systolic Array Data Flows for Efficient Matrix Multiplication in Deep Neural Networks," Oct. 2024, doi: <https://doi.org/10.48550/arXiv.2410.22595>.